

3.4 Comparison with Fuzzing

Antero Morgado¹¹⁹²¹³, António Palma¹¹⁹²⁰³ and Vasco Conceição¹⁰⁶⁴⁸¹

Professor João Pereira

Professor José Fragoso

This manuscript was compiled on May 15, 2026

Abstract—In this exercise we compare the concrete test suites produced by AFL++ fuzzing (Exercise 2) and KLEE symbolic execution (Exercise 3) when applied to the same `TreeTable` implementation. Both tools targeted the same four functions: `treetable_add`, `treetable_get`, `treetable_get_first_key`, and `treetable_get_greater_than`. We evaluate each technique along four dimensions: coverage achieved, bugs detected, test-generation cost, and effort required to write the harness or symbolic tests.

Keywords—fuzzing, symbolic execution, AFL++, KLEE, concrete tests, coverage

1. COVERAGE ACHIEVED

Both fuzzing (AFL++) and symbolic execution (KLEE) achieved near-optimal results, reaching 100% coverage in `treetable_add`, `treetable_get`, and `treetable_get_greater_than`. For the `treetable_get_first_key` function, both techniques failed to cover a single specific line due to an edge case condition not triggered by the input generators.

The high coverage in both methods demonstrates that for this specific `TreeTable` implementation, the state space is well-explored by both feedback-driven mutations (fuzzing) and constraint solving (symbolic execution).

- `treetable_add`: Full code coverage achieved by both tools.
- `treetable_get`: Full code coverage achieved by both tools.
- `treetable_get_first_key`: Nearly full coverage, with one line remaining uncovered by both techniques.
- `treetable_get_greater_than`: Full code coverage achieved by both tools.

2. BUGS DETECTED

Significant differences were observed between the two techniques. While KLEE (symbolic execution) did not report any bugs within the explored search space, AFL++ (fuzzing) successfully identified several critical vulnerabilities:

- **Segmentation Fault in Empty Lookups**: AFL++ discovered that calling `treetable_get_first_key` on an empty tree causes a crash because `tree_min` dereferences the sentinel node without a size check.
- **Logic Invariant Violations**: Using the balanced and sorted oracles, the fuzzer identified specific insertion sequences where `treetable_add` and its rebalancing logic failed to maintain Red-Black Tree properties.
- **Memory Management Issues**: A medium-severity bug was found where `treetable_add` fails to check for NULL after a memory allocation call.
- **API Inconsistencies**: Testing revealed that certain functions declared in the header (like `treetable_get_last_key`) were missing implementations in the source file.

The failure of KLEE to find these bugs, despite achieving similar coverage, may be attributed to the complexity of the symbolic constraints required to trigger deep logic errors in a balanced tree compared to the high-throughput random mutations of AFL++.

3. TEST GENERATION COST

The test generation cost reveals a significant trade-off between execution time and computational depth. Fuzzing with AFL++ required a

fixed, high-intensity commitment of 30 minutes of continuous CPU time to explore the program's state space through high-frequency mutations. This process yielded a large volume of initial inputs that necessitated a subsequent minimization step with `afl-cmin` to reduce the corpus to a manageable size while maintaining coverage. While computationally expensive in terms of duration, the process was fully automated and successfully uncovered deep logic bugs and edge-case crashes that simple execution might miss.

In contrast, symbolic execution with KLEE followed a more analytical path, where the generation cost was characterized by the time required for the solver to explore different execution branches. This method generally produces a smaller, more focused test suite where each test case is mathematically derived to exercise a unique path. However, the efficiency of KLEE is highly dependent on the complexity of the constraints; in this case study, while KLEE was faster at generating coverage-equivalent tests, it failed to explore the specific invalid states that AFL++ reached through its approach.

4. EFFORT TO WRITE

The effort required to implement both techniques was comparable, though they demanded different analytical focuses. For the AFL++ harness, the primary challenge lay in designing a robust command decoder to map raw input bytes from standard input to the public `TreeTable` API functions. This required careful handling of data lifetimes to ensure that keys and values remained valid during operations. On the other hand, writing the symbolic tests for KLEE involved a more conceptual effort in property-based testing. Instead of just feeding data, it was necessary to define symbolic constraints and determine which sequences of operations would best exercise the tree's invariants, such as checking for the preservation of balance and sorting order after modifications. Notably, the use of AI assistants significantly streamlined these tasks, helping to automate the generation.

5. CONCLUSION

In conclusion, for this specific case study, fuzzing with AFL++ first would lead to better results. Although both techniques achieved nearly identical high coverage, the fuzzer was significantly more effective at uncovering critical bugs—ranging from segmentation faults to complex logic violations—that the symbolic engine missed. Fuzzing proved to be a "low-effort, high-reward" approach; once the harness was implemented, the tool's stochastic nature explored edge cases that were not immediately obvious when designing symbolic constraints. While symbolic execution remains a powerful tool for formal verification, the immediate feedback and bug-finding depth provided by AFL++ make it the superior first choice for testing real-world C data structures.